THE EDITOR MANUAL

bm SDK

Projects, templates, maps and building a game, on the console.

Contents

What bm SDK is 4

What it connects to 4

The rules of the suite 4

Opening bm SDK 5

The screen 6

The project page 6

The other pages 6

Creating a project from a template 7

Tutorial: Empty 2D 7

Tutorial: Platform 2D 8

Tutorial: Top-down 2D 8

Tutorial: Shooter 2D 8

Tutorial: Versus 2D 9

Tutorial: Online 2D 9

Tutorial: 3D scene 9

Tutorial: 3D with models 9

Tutorial: Stages 2D 9

The map editor 11

What it does 11

The terrain brush (t) 11

Good to know 12

Caveats 12

3D maps and level tools 12

Try it, step by step 13

Budgets and the dev kit report 14

What it shows 14

In the game 14

Try it, step by step 14

Building and exporting **15**
What it does 15
Good to know 15
Caveats 15
Try it, step by step 15
The RGB30 edition 15
The sounds page (RGB30) 16

Keys **17**

Tips and troubleshooting **20**

Open points **21**

README

What this is. The manual of bm SDK, the hub of the bm suite: where a game starts, where you try it, measure it and build it.

How to use it. Read chapters 1 to 3 once. Then follow the walk-through of the template you want in chapter 4. Chapter 8 is the table of keys.

What is missing. Every fact not confirmed by the repository is marked with a highlighted TO FILL tag.

What bm SDK is

bm SDK is the program where a game starts. It holds the project's code, sprites, map and 3D models in one file, tells you how big and how fast the game is, tries it, and builds the final game.

What it connects to

- **The other tools of the suite.** On the project page, keys `1` to `6` open the same file in bm Code, bm Pixel, bm Studio, bm Animator, bm Mesh and bm Sound. Their menu has **Back to bm SDK**, which saves and returns.
- **The Market.** The suite's apps can be installed from the Market Apps page. The SDK prefers the copies installed on the SD card over the ones built into the kernel, so tools can be updated from the Market.
- **The Lib tab.** The menu's Lib tab lists the resources found in your games (models, images, sounds, maps, palettes, kits). Its options include *Copy into a project...*, which brings a resource into a project. The tab is hidden by default: `lib_tab=1` in `bm/config.txt` brings it back.
- **The Games tab.** A game you build appears there.

The rules of the suite

BEWARE

The editors modify only **projects** (`.bme`). A **game** (`.bm` , `.b16`) is read-only. If you save while a game is open, the console asks *Save an editable copy: name.bme?* and from then on you work on the copy.

- **Ctrl+B** builds the game of the project: a `.bm` , or a `.b16` when the target is `.b16` .
- **Ctrl+Shift+B** exports the `.b16` (the RGB30 edition).
- A `.b16` never replaces the `.bm` : they are two editions, with their own builds.

Opening bm SDK

You can start it three ways:

1. From the **Dev** tab of the menu: choose **bm SDK**.
2. From a game: press **X** on its cover and choose *Open in the SDK*. The game opens as a read-only game; the first save makes its editable copy.
3. From the monitor: `e`.

On the RGB30 the program is **bm SDK (RGB30)**: see chapter 7 for what is different.

The screen

The SDK looks like bm Studio and bm Animator: a panel on the left with lists, two lines above the view with the name, the keys at the bottom. The pages are chosen with the F keys (on a gamepad: Y + left/right).

Key	Page
F1	The project; F1 again: the dev kit
F2	The code
F3	2D: the sprites; F3 again: the map
F4	3D: models and animations
Esc	The menu

[TO FILL: shots/editor-sdk-project.png]

The project page

The left panel has two groups.

- **OPEN IN**: `1` bm Code, `2` bm Pixel, `3` bm Studio, `4` bm Animator, `5` bm Mesh, `6` bm Sound.
- **PROJECT**: `t` the title, `a` the author, `r` the screen, `b` the target (`.bm` or `.b16`), `n` a new project from a template.

Move with up and down, do with Enter, or press the key of the row. The key `r` goes through the sizes `640x360` , `480x270` , `320x180` , `256x256` , `360x360` and back to `640x360` .

On the right, **CONTENTS** shows what is in the file: code (lines, KiB, tokens), sprites (how many cells are drawn), map (size, layers, cells), models, animation, sound and the tiles with flags. **LAST TRY** shows the numbers of the last F5. **ASSISTANT** says that F6 gives step-by-step guides to make a 2D or a 3D game.

The other pages

- **F2, the code.** A quick editor with syntax colours and the number of tokens in the status line. For more than a quick change, press `1` on the project page to open bm Code. The help on an API function (**Ctrl+Q**) and the check of unknown API names on save are in bm Code, not in this quick editor.
- **F3, the sprites.** The sheet, the palette and the **flags** of the chosen cell (keys `0` to `7`). The flag convention is bmlib's: 0 wall, 1 platform, 2 ladder, 3 water, 4 hurts.
- **F3 again, the map.** See chapter 5.
- **F4, the 3D.** The project's models in a list and the animations of the chosen one. Key `i` writes into the code the lines to load and draw the model.
- **F6, the assistant.** On the project page it gives the guides to make a game; on the other pages it helps with code, sprites and 3D models. Hold **F12** to see the keys of the page you are on.

Creating a project from a template

Start a new project with **Ctrl+N**, with *New project...* in the menu (Esc), or with `n` on the project page. A list of templates opens: choose one with up and down and Enter. Each template brings the code, the sprites and the map it needs, and **F5** tries it right away.

Template	What it is
Empty 2D	a ball that moves: the smallest game
Platform 2D	run and jump on the map, the camera follows
Top-down 2D	walk among walls and pick up the coins
Shooter 2D	fly, fire, waves of enemies, game over
Versus 2D	two players on one console: punches with hitboxes
Online 2D	two to four consoles on the network: a lobby and lockstep
3D scene	floor, cubes, a ball with its shadow, a camera that follows
3D with models	the models of bm Studio, animated (bm Animator)
Stages 2D	a title, three stages, an ending and a saved best time

A new project is not saved yet: the project page says `not saved yet` until you use **Save as**. If you have unsaved changes, the SDK says `unsaved changes: choose the template again to leave them` : choose the template again to confirm.

Tutorial: Empty 2D

The smallest game: a ball you move with the arrows.

1. Start bm SDK (chapter 2). You see the project page.
2. Press **Ctrl+N**. A list of templates opens.
3. Choose **Empty 2D** with up and down and press **Enter**. The status line says `new project (Empty 2D): Esc > Save as gives it a name, F5 tries it` . The title is `New game` .
4. Press **F2**. You see the code of the game: `_init` , `_update` and `_draw` .
5. Move the cursor with the arrows to the line `print("hello!", ...)` and change the word `hello!` to your own text.
6. Press **Esc**, choose **Save as...** and press Enter. A box asks `file name (8.3, in /bme)` and suggests `mygame.bme` . Type a name and press **Enter**. The status line says `saved` and the name of the file.
7. Press **F5**. The SDK saves and the console plays the game: a yellow ball on a dark background with your text. The arrows move the ball.
8. Leave the game. Press Start + Select, Ctrl+Shift+Esc or Ctrl+Esc (Esc alone goes back inside a tool).
9. You are back in the SDK. Press **F1**: the **LAST TRY** line now shows fps and milliseconds.

[TO FILL: shots/editor-sdk-empty.png]

TRY IT

Change the number `12` in `circfill(x, y, 12, 0xFFD050)` to make the ball bigger or smaller, press F5 again and see the difference.

Tutorial: Platform 2D

A hero that runs and jumps on a map with ground and planks. The camera follows it.

1. Start bm SDK. You see the project page.
2. Press **Ctrl+N** and choose **Platform 2D** with Enter. The project has a hero, ground tiles, planks and a map already built.
3. Press **F3**. You see the sprite sheet. Sprite 1 is the hero, 2 the ground, 6 the planks.
4. Choose the ground cell on the sheet (`Tab` chooses on the sheet). At the bottom the **FLAGS** row shows flag `0` lit: the ground is solid. For the planks, flag `1` is lit: a platform, solid from above only.
5. Press **F3** again. You see the map: a floor of ground tiles with two gaps and six platforms of planks. The status line shows the cell, the tile and the layer.
6. Press **F2** to read the code. It uses `bmlib` : `lib.step(hero)` does gravity and collisions with the map, `lib.camera` follows the hero.
7. Press **F3** until the map shows. Move the cursor to an empty cell in the air, choose the planks with `x` on a plank, and press **Space** to place a new plank.
8. Press **Esc**, **Save as...**, type a name and press **Enter**.
9. Press **F5**. The hero stands on the ground. The arrows run, **act1** (Space on the keyboard; the screen shows the button) jumps, **down** drops through a plank.
10. Run to the first gap: if you fall, the hero comes back to the start. Find your new plank.
11. Leave the game and you are back in the SDK. See step 8 of Empty 2D for the keys.

[TO FILL: shots/editor-sdk-platform.png]

TRY IT

Press `z` on the map to switch to the 16x16 brush, or place a plank higher up and check that the hero can still reach it.

Tutorial: Top-down 2D

1. Press **Ctrl+N** and choose **Top-down 2D**.
2. Press **F3** twice to see the map: walls around the area and coins in it. Walls have flag `0` (solid).
3. Press **Esc**, **Save as...**, type a name and press Enter.
4. Press **F5**. The arrows walk the hero; touching a coin picks it up and plays a note. The top left shows `coins` and the count.

Tutorial: Shooter 2D

1. Press **Ctrl+N** and choose **Shooter 2D**.
2. Press **F3** to see the two sprites: the ship (1) and the enemy (5).

3. Press **Esc**, **Save as...**, type a name and press Enter.
4. Press **F5**. The arrows fly the ship, **act1** (Space on the keyboard) fires; the enemies come from the right. When the game is over, **act1** starts again.

Tutorial: Versus 2D

1. Press **Ctrl+N** and choose **Versus 2D**.
2. Press **Esc**, **Save as...**, type a name and press Enter.
3. Press **F5**. Two fighters: each with a controller; alone, the computer plays the second. Left and right walk, **act2** (X on the keyboard) punches, **act1** (Space) jumps. The punches are hitboxes and the bodies hurtboxes.

Tutorial: Online 2D

1. Press **Ctrl+N** and choose **Online 2D**.
2. Press **Esc**, **Save as...**, type a name and press Enter.
3. Press **F5** on two to four consoles on the same network. **A** hosts a match or joins one that was found; the host starts it with **Start**. Each console moves its square.
4. In the code, change the game's tag `MYG1` in `net.open` to a tag of your own.

> **NOTE**
> The game's random numbers come from the match's seed (`lib.rng`), never from `math.random` . Over the internet a relay is needed: see the `relay` option of `net.open` .

Tutorial: 3D scene

1. Press **Ctrl+N** and choose **3D scene**.
2. Press **Esc**, **Save as...**, type a name and press Enter.
3. Press **F5**. A floor, four cubes and a ball with its shadow. The arrows move the ball, **act1** (Space on the keyboard) jumps, and the camera follows. The top left shows the number of triangles.

Tutorial: 3D with models

1. Make or open models first: press **F4** and see the list of the project's models, or press `3` on the project page to open bm Studio.
2. Press **Ctrl+N** and choose **3D with models**.
3. Press **Esc**, **Save as...**, type a name and press Enter.
4. Press **F5**. The models of the project turn on a stand; with a skeleton (bm Animator) they play their first animation. Left and right change the model, up and down the zoom. With no model, a cube is shown.

Tutorial: Stages 2D

A whole small game: a title screen, three stages, an ending and a saved best time. The hero runs and jumps like in Platform 2D.

1. Press **Ctrl+N**. *Stages 2D* is the last row of the list: press **up** once from the first row, then **Enter**. The status line says `new project (Stages 2D)` .

2. Press **Esc**, **Save as...**, type a name and press **Enter**.
3. Press **F3** twice to see the map. The three stages are three bands of 48 rows one under the other (rows 0-47, 48-95, 96-143). In each band the ground has gaps, there are planks, and the exit (tile 7, a flag) is at the right.
4. Press **F5**. The title shows **NEW GAME**; up and down choose, **A** starts. Once you have reached stage 2 or 3, a second row **CONTINUE STAGE** appears.
5. Play: left and right walk, **A** jumps, **down** drops through a plank. The top left shows the stage, the lives and the time.
6. Fall down a gap: you lose a life (you have 3) and the stage starts again. With no lives left, **GAME OVER** appears and **A** goes back to the title.
7. Reach the flag: `STAGE 1 CLEAR`, then **A** for the next stage. After stage 3 the ending shows `ALL CLEAR` with your time and the best time. Only a game started with **NEW GAME** can set the best time.

NOTE

The game keeps the best time and the stage you reached with `save()`. How saves work is in the API manual.

TRY IT

Move the flag of stage 1 closer to the start (a tile 7 on the map), or add a plank over the first gap, and press F5 again.

The map editor

Press **F3** to see the sprites, then **F3** again for the map. A map is made of tiles from the sprite sheet, on layers.

[TO FILL: shots/editor-sdk-map.png]

What it does

- A map has up to 8 layers. `l` goes to the next layer, `Shift+L` adds a new one (`layer2` , `layer3` ...), `o` shows this layer only or all of them. The status line shows the layer and its name, and the call to draw it: `map(..., "name")` .
- `Space` places the tile, `Backspace` clears the cell, `x` picks the tile under the cursor, `f` fills, `u` undoes.
- `,` and `.` take the tile before or after; `Tab` opens the tile chooser.
- `z` switches between 8x8 and 16x16: a 16x16 tile is 2 x 2 cells, placed on even cells.
- `+` and `-` zoom in and out; `m` shows the whole 256x256 map on one screen (*Overview*).
- `c` shows the flags of the tiles.
- `t` turns the terrain brush on and off (see below).
- `Home` , `End` , `PgUp` and `PgDn` move a page left, right, up and down.

The terrain brush (t)

The terrain brush paints land, water and walls and chooses the border tiles for you. Press **t** on the map page (menu: *Terrain brush*). Press **t** again to leave. It does not work in the 3D view.

A terrain set is a block of 4 x 4 tiles of the sprite sheet. The tile you chose is the first tile of the block, at the top left. In 8x8 mode the 16 tiles are next to each other. In 16x16 mode (`z`) each tile is 2 x 2 cells, so the block steps by 2 cells.

For each tile of the set, bm SDK looks at the four neighbours. Each neighbour that is in the set adds a number: north 1, east 2, south 4, west 8. The sum is the mask, from 0 to 15. The tile in the block is at the column `mask % 4` and the row `mask // 4` , counted from the first tile:

Row	Column 0	Column 1	Column 2	Column 3
0	mask 0: alone	mask 1: N	mask 2: E	mask 3: N+E
1	mask 4: S	mask 5: N+S	mask 6: E+S	mask 7: N+E+S
2	mask 8: W	mask 9: N+W	mask 10: E+W	mask 11: N+E+W
3	mask 12: S+W	mask 13: N+S+W	mask 14: E+S+W	mask 15: all four

Draw the 16 tiles of the set in this order on the sheet. Then:

- **Space** paints the cell with the set. The cell and its 4 neighbours take the tile that fits their mask. Only the neighbours that are already in the set change.
- **Backspace** erases the cell and fits its 4 neighbours again.
- Each stroke works on the current layer. One stroke is one undo step (**u**).

The status line shows `TERRAIN base N` while the brush is on. The hint bar shows the chip `t terrain` . The key **m** shows the whole map: it is in the help (**F12**). The bar ends with the chip F12 keys.

Click it, or hold F12, to see every key of the page.

The RGB30 SDK has the same brush. Press **t** on the map page, or choose *Terrain brush* in the **X** list. The status line shows `TERRAIN` and the base tile. Paint and erase work as on the Pi. The hint chip **c** (flags) became **t** (terrain) on a keyboard; **c** stays in the help and the menu.

Good to know

- Map cells are numbers: 0 is empty, the others are the tile.
- With the mouse: the left button places the tile, the middle one picks it, the right or middle button dragged moves the view, the wheel scrolls.
- In the code page, drag the left button to select text and double click to select a word; Ctrl+C and Ctrl+X use the selection, and a letter or Backspace replaces it. In a dialog line, a click puts the caret there. On the RGB30 sounds page the wheel over a field or a step changes it, a double click plays it, and a click on a long message hides it.
- When you go to a layer named `objects`, the brush becomes 16x16 and the first placeholder tile (1024 or more) is chosen. Placeholders are tags the game does not draw: the game reads them and builds things there.
- The flags belong to the **tile** on the sheet, not to a place on the map. Set them on the sprite page with keys `0` to `7`.

Caveats

- A map has at most 8 layers: the SDK says `a map has at most 8 layers`.
- The menu entry *New map layer (n/8)* does the same as `L` and shows how many you have.
- **Import a Tiled map**: save the map in Tiled as JSON (`.tmj`): orthogonal, tiles of 8x8 or 16x16 pixels, layer format CSV, fixed size, at most 256x256 cells. Put the file in the `/docs` folder of the SD card. Open the menu (`Esc`), choose *Import a Tiled map (.tmj)...*, choose the file, then *add the layers to the map* or *replace the map*. Each Tiled tile layer becomes a map layer with the same name. A Tiled tile is the sheet tile in the same column and row (a 16x16 tile is 2x2 cells). Tile properties `solid`, `platform`, `ladder`, `water` and `hurt` (boolean) turn on flags 0 to 4. Object layers are ignored. The import cannot be undone: reopen the project to go back.

3D maps and level tools

Press **v** on the map page to toggle between 2D editing and real-time **3D preview**.

- **Orbital camera**: arrow keys rotate (left/right yaw) and zoom (up/down distance); **Shift+Up** and **Shift+Down** adjust the tilt pitch.
- **Template: 3D map**: press **T** (or choose it from the menu) to automatically configure the four canonical 3D layers: `floor`, `height`, `walls`, and `objects`.
- **Rename layer**: press **r** to rename any layer with a dialog prompt.
- **Path tool (`Shift+P`, `Shift+W`)**: press `Shift+W` to cycle corridor width from 1 to 5 tiles. Press `Shift+P`, navigate to the start cell and press Enter, then navigate to the destination cell and press Enter: a corridor of width *N* is drawn on the `floor` layer with the active tile.
- **Apples row tool (`Shift+A`)**: press `Shift+A`, press Enter at the start cell and Enter at the destination cell: a continuous line of apple placeholders (ID 1026) is placed on the `objects` layer.
- **Place crates and lamps (`Shift+C`, `Shift+O`)**: `Shift+C` places a crate placeholder (ID 1028) at the cursor cell on the `objects` layer, and `Shift+O` places a lamp placeholder (ID 1032).

- **Place the start and the goal (`Shift+S` , `Shift+G`)**: `Shift+S` places the start placeholder (ID 1024) and `Shift+G` the goal (ID 1030) at the cursor cell.
- **Status bar**: displays the current counts of apples, crates, and lamps, together with an estimated triangle budget (floor + objects).
- **Undo**: `u` or `Ctrl+Z` undoes an entire path, row, or object placement in one step.

Try it, step by step

1. Open the Platform 2D project (chapter 4).
2. Press **F3** twice. You see the map.
3. Press **m**. The whole map appears on one screen with a box that shows the part you were looking at.
4. Press **m** again to return to the normal view.
5. Press **z** and place a tile with **Space**. The tile is now 2 x 2 cells.
6. Press **u** to undo it.

Budgets and the dev kit report

Press **F1** on the project page, then **F1** again. The second page is the **dev kit**.

What it shows

- **CODE**: tokens, lines, KiB, number of functions and the biggest ones.
- **DATA IN MEMORY WHILE IT RUNS**: sheet, map, models, skeletons, sound and z-buffer, and the total. The Lua comes on top. This is the same sum as `stat(13)` .
- **CARTRIDGE FILE**: the size of the saved file against the mark of 100 MiB, the most the Market takes for one file. The mark is only a guide: the build does not refuse a larger file and the kernel has no size cap. Before the first save it says `not saved yet: Save as, then the size of the file` .
- With the target `.b16` : the lines of the code that the `.b16` will not have (`math.random` , `time()` , files...). With `.bm` : `.bm has no cap` .
- **LAST TRY (F5)**: fps, average and maximum milliseconds of a frame, frames over 16.7 ms, the peak RAM (Lua and data), the instructions of the heaviest frame, tokens, triangles, and whether the 3D was drawn on the GPU or the ARM.

[TO FILL: shots/editor-sdk-devkit.png]

In the game

- `stat(1)` ms, `stat(2)` fps, `stat(11)` tokens, `stat(12)` Lua KiB at most, `stat(13)` KiB of data.
- **F11** in any game shows the overlay: fps, ms, the CPU and GPU bars.

Try it, step by step

1. Open a project and press **F5**.
2. Come back to the SDK and press **F1**: **LAST TRY** shows the numbers.
3. Press **F1** again: the dev kit shows the memory and the size of the file.
4. Press **b** to change the target to `.b16` and see what the RGB30 edition would not accept.

Building and exporting

What it does

- **Ctrl+B**, or *Build the game .bm* in the menu: builds the game of the project. When the target is `.b16`, it builds the `.b16` and the menu entry says so.
- **Ctrl+Shift+B**, or *Export .b16*: exports the `.b16` whatever the target.
- A project is in `/bme`; its game is written in the games folder: `/carts` on the Pi, `/bm` on the RGB30. The **Games** tab lists that folder. A project found next to its game on an older card builds next to it.
- On success the SDK says `built /carts/name.bm: it is in Games` (the path of the game).

Good to know

- A game is built from its project. If a game is open, the SDK saves its editable copy first.
- Setting the target to `.b16` (key `b` on the project page) changes the screen to `360x360` if it was not square, and shows the size of the file against the 100 MiB mark. The mark is a guide, not a limit.
- The target is remembered for each project file.
- On the RGB30, exporting a `.b16` from `/carts` also copies it to `/bm/`, so it appears at once in Games. On the RGB30 the Games tab lists `/bm/` and `/carts`; if the game is written somewhere else, the status line adds `(not in the Games tab)`.

Caveats

- `build: a game is built from its project (save it first)` appears when the file is not a project yet: use **Save as**.
- Building again replaces the game.

Try it, step by step

1. Open or create a project and save it with **Save as**.
2. Press **Ctrl+B**.
3. Wait for `built /carts/name.bm: it is in Games`.
4. Leave the SDK (Esc, **Exit bm SDK**) and open the **Games** tab: your game is there.

COPY AND PASTE

```
Ctrl+B          build the game (.bm, or .b16 when the target is .b16)
Ctrl+Shift+B    export .b16
```

The RGB30 edition

bm SDK (RGB30) is a separate edition for the 360x360 screen and the pad. It is in the **Dev** tab and in the game's options (**X** on its cover in Games: *Open in bm SDK*).

- The pages are the same as on the Pi, laid out again for 360x360.
- The pad: **L1 / R1** change page, **Start** opens the menu, **X** opens the list of the actions of the page, **L2 / R2** the previous or next colour or tile, **ok** and **back** are B and A (`confirm=a` swaps them).
- Text is typed with the on-screen keyboard (`padtype`); a keyboard works as on the Pi.
- A new project is 360x360 with the target `.b16`.

- The templates are the same, *Stages 2D* included.
- **Try**: Start + Select (or Ctrl+Esc) leaves the game and the SDK comes back.
- The programs bm Code, Pixel, Studio, Animator, Mesh and Sound are not on the RGB30, so *Open in ...* is not offered.

> **BEWARE**
> The pad, the screen and the sound of bm SDK (RGB30) were driven by keyboard in an emulator (QEMU) and by the repository tests only. They are **not verified on a real RGB30**.

- **[TO FILL: bm SDK (RGB30) tried on a real console with the pad alone; whether it is published in the Market, the repository files disagree]**

The sounds page (RGB30)

Press **F7** for the page of the project's sounds and effects (on the pad it is the page after *3D*, reached with R1). It works on the sound bank saved in the project. **F7** again, or **Y + up/down**, goes between the two views.

> **BEWARE**
> The sounds page was driven in an emulator and by the repository tests only. It is **not verified on a real RGB30**.

Sounds

The left list has 32 sound slots, the right panel the fields of the chosen sound: **WAVE**, **VOLUME**, **DUTY**, **ATTACK**, **DECAY**, **SUSTAIN**, **RELEASE**.

- Up and down choose the field, left and right change the value; **PgUp / PgDn** (on the pad: hold left or right) change it 8 at a time.
- `[` and `]` (or `,` and `.` ; on the pad **L2 / R2**) choose the sound.
- **Enter** (on the pad **ok**) plays the sound on the note A4.
- `c` clears the sound.
- `i` opens *Sounds from a file (Lib)*: first the list of the `.bm` and `.bme` files with a sound bank (from `/bme` , `/carts` , `/bm` , the root of the card and `/bm/lib`), then the sounds in the file you chose. The sound goes into the first empty slot; the status line says `sound N taken from the file` . If there is none, it says `no .bm or .bme file with sounds` or `no empty sound slot` .

Effects

The effects view has 64 effects of 32 steps each. A step is a note, the sound that plays it and its volume.

- Left and right choose the step, up and down change its note (a lower case letter is a sharp).
- **Enter** on an empty step puts a note (the last note, with the sound chosen in the sounds view) and plays it; on a full step it plays it.
- **Backspace** (on the pad **back**) clears the step.
- `+` and `-` change the speed of the effect by 10 ms (from 10 to 500 ms). `p` plays the whole effect.

The changes are saved with the project (**Ctrl+S**).

Keys

Hold **F12** in the SDK to see the keys of the page you are on.

Key	Where	What it does
F1	anywhere	the project; again: the dev kit
F2	anywhere	the code
F3	anywhere	the sprites; again: the map
F4	anywhere	3D models and animations
F7	anywhere (RGB30)	the sounds; again: the effects
Esc	anywhere	the menu
Ctrl+N	anywhere	a new project from a template
Ctrl+O	anywhere	open a cartridge
Ctrl+S	anywhere	save
Ctrl+Shift+S	anywhere	save as
Ctrl+B	anywhere	build the game of the project
Ctrl+Shift+B	anywhere	export the .b16
F5 or Ctrl+R	anywhere	try the game
F6	anywhere	the assistant
F9	anywhere	the assistant explains the last error
1 - 6	project	open in bm Code, Pixel, Studio, Animator, Mesh, Sound
t / a	project	title / author
r	project	the screen size
b	project	target .bm or .b16
n	project	a template
Ctrl+G	code	go to the error of the last try
Ctrl+X or Ctrl+K	code	cut the line
Ctrl+C / Ctrl+V	code	copy the line / put it above
Ctrl+D	code	duplicate the line
Space / Backspace	sprites	draw / erase
x	sprites	pick the colour
f	sprites	fill
z	sprites	8x8 / 16x16

Key	Where	What it does
h / v	sprites	flip
u	sprites	undo
0 - 7	sprites	a flag of the cell
Space / Backspace	map	place the tile / clear the cell
v	map	toggle 2D / 3D preview
Shift+P / Shift+W	map	path tool (start, end: Enter) / width (1-5)
Shift+A	map	apples line tool (start, end: Enter)
Shift+C / Shift+O	map	place crate (1028) / place lamp (1032)
Shift+S / Shift+G	map	place the start (1024) / the goal (1030)
r	map	rename layer
T	map	template: 3D map layers
x	map	pick the tile
f	map	fill
Tab	map	choose the tile
z	map	8x8 / 16x16
+ / - / m	map	zoom in / out, overview
t	map	terrain brush on / off
l / Shift+L	map	next layer / a new one
o	map	this layer only / every layer
c	map	show the flags
u	map	undo
up / down	3D	the model
left / right	3D	the animation
Space	3D	play, pause
q / e, w / s	3D	turn, tilt
i	3D	the code that draws the model
+ / -	3D	zoom
r	3D	turn by itself
Enter, a, m	3D	open in bm Studio, bm Animator, bm Mesh

Key	Where	What it does
up / down, left / right	sounds (RGB30)	the field, the value
[/] or , / .	sounds (RGB30)	the sound before / after
Enter	sounds (RGB30)	play (on an empty effect step: put a note)
c / i	sounds (RGB30)	clear the sound / sounds from a file (Lib)
Backspace	effects (RGB30)	clear the step
+ / - / p	effects (RGB30)	speed of the effect / play the effect

Tips and troubleshooting

- **F5 asks for a name.** A project that has no file yet must be saved first: F5 opens the **Save as** box; choose a name and press Enter. The message `no name yet: use Save as` appears only when you use plain **Save** on a project with no name.
- ** `unsaved changes: choose again to confirm` .** Leaving, opening or starting a new project with unsaved changes asks twice: repeat the action to confirm.
- ** `no .bm or .bme files on the SD card` .** *Open...* found nothing in `/bme` , `/carts` or in the root. It lists `.bm` , `.bme` and `.b16` files.
- ** `cannot measure: ...` on the project page.** The SDK could not read the numbers of the file; save it and look at the message.
- ** `no error to explain` . **F9 works only after a try that ended with an error.** Ctrl+G** in the code goes to the line.
- **My game is slow.** Look at the dev kit after a try: frames over 16.7 ms, the heaviest frame and the memory tell you where.
- **A `.b16` will not have some things.** With the target `.b16` , the dev kit lists them by line.
- **A big map.** A big map of 16x16 tiles is best made by a script and retouched in the SDK, with `z` and the overview `m` .
- **Where is my game?** In the **Games** tab. If it is not there, check the message after Ctrl+B.

Open points

- **The screen**: picture to take: shots/editor-sdk-project.png
- **Creating a project from a template**: picture to take: shots/editor-sdk-empty.png
- **Creating a project from a template**: picture to take: shots/editor-sdk-platform.png
- **The map editor**: picture to take: shots/editor-sdk-map.png
- **Budgets and the dev kit report**: picture to take: shots/editor-sdk-devkit.png
- **Building and exporting**: bm SDK (RGB30) tried on a real console with the pad alone; whether it is published in the Market, the repository files disagree